2026 IEEE 3rd International Conference on Big Data Science and Engineering (ICBDSE) | 979-8-3315-5243-5/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICBDSE70225.2026.11635372

Ghost-YOLOv11: A Lightweight and Efficient Deep Learning Model for Real-Time Mango Counting

1st Minh Ngoc Le*
Faculty of Information Technology
FPT University
CanTho, Vietnam
minhle5823@gmail.com

2nd Bao Quoc Tien
Faculty of Information Technology
FPT University
CanTho, Vietnam
tienquocbao@gmail.com

3rd Duy Thanh Huynh
Faculty of Information Technology
FPT University
CanTho, Vietnam
hthanhduyy@gmail.com

4th Luong Vuong Nguyen
Faculty of Artificial Intelligence
FPT University
DaNang, Vietnam
vuongnl3@fe.edu.vn

5th Hoang Ngoc Tran
Faculty of Information Technology
FPT University
CanTho, Vietnam
hoang2531992@gmail.com

***Abstract*—Reliable mango counting in orchards requires a detector that can separate clustered fruit, tolerate leaf occlusion and changing light, and still run within an edge-computing budget. We address this setting with Ghost-YOLOv11n, a lightweight version of YOLOv11n built for image-level mango counting. Instead of altering the prediction head or the multi-scale detection route, the model replaces selected feature-generation blocks with GhostConv and C3k2Ghost modules. This choice keeps the comparison centered on computational redundancy inside the feature path. On MangoYOLO, Ghost-YOLOv11n uses 1.85M parameters and 2.70 GFLOPs, whereas the YOLOv11n baseline uses 2.59M parameters and 3.22 GFLOPs. With the validation-selected operating point of confidence 0.55 and NMS IoU 0.25, the proposed model records 0.773 MAE and 422.9 FPS on the test split. The ablation results show that modifying C3k2-style aggregation is the main source of the counting and efficiency gains.**

***Index Terms*—Mango Counting, Edge Object Detection, GhostConv, C3k2Ghost, Counting Calibration.**

I. Introduction

Yield estimation becomes more useful when image processing can be performed close to the orchard instead of after data are transferred to a remote workstation [1]–[3]. Mango images collected in the field are still challenging for a compact detector: fruit may touch or overlap, leaves can hide part of the target, illumination varies across the canopy, and background areas may have colors close to the fruit surface [4]–[6]. In crowded regions, these visual conditions can produce boxes that are poorly separated or duplicated, which directly changes the final count [7]. A counting-oriented detector therefore needs both efficient feature extraction and a post-processing setting that is not tuned on the test images.

This paper evaluates a constrained redesign of YOLOv11n for mango counting. Following the GhostNet principle of generating part of the feature maps through inexpensive transformations [8], we replace selected YOLOv11n Conv and C3k2 components with Ghost-based counterparts [9]. The detection head, feature-fusion route, and training protocol are left unchanged so that the study measures the effect of lighter feature generation. The contributions are summarized as follows:

(1) **A compact YOLOv11n counting model**: GhostConv and C3k2Ghost are introduced into the YOLOv11n feature path while the first Conv layer, SPPF, C2PSA, and Detect layers remain standard.
(2) **A fixed counting protocol**: The confidence and NMS IoU values are chosen on the validation split and then reused for every test-set comparison.
(3) **An empirical MangoYOLO study**: On MangoYOLO [10], the proposed model reduces YOLOv11n parameters by 28.6% and reaches 0.773 MAE with real-time inference speed.

II. Related Work

A. Mango Counting in Field Imagery

Field fruit counting differs from generic detection because each missed or duplicated box changes the estimated yield. Mango scenes often contain leaf cover, dense bunches, non-uniform sunlight, and fruit-background color ambiguity [11], [12]. Heavy detectors can improve representation capacity, but their memory and arithmetic costs are difficult to justify for drones, embedded boards, or on-farm handheld devices [13]. Existing agricultural detectors have addressed this issue by adopting compact backbones, GhostNet-like operators, attention blocks for hidden targets, or post-processing strategies that improve count stability [14]–[17]. These studies motivate a detector that is small enough for deployment while being evaluated with a counting-specific threshold policy.

B. Motivation for Ghost-Based Replacement

Convolutional feature maps may contain repeated responses, especially when neighboring objects have similar texture and color [18]. This is a natural fit for Ghost-style modules, which compute a subset of informative maps with regular convolution

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 19:25:00 UTC from IEEE Xplore. Restrictions apply.

and synthesize additional maps using cheaper operations. Our study applies this idea to YOLOv11n by modifying Conv and C3k2 transformations while leaving the detector head intact. All models are then compared under one validation-chosen confidence/NMS setting, so the reported counting results do not depend on per-model test-set threshold tuning.

III. Methods

YOLOv11n is selected as the reference network because, in our implementation, it produced the lowest counting error among the nano YOLO baselines while using fewer parameters than YOLOv8n (Table I). It also offers a clean replacement target: the Conv and C3k2 units can be substituted without changing the Detect layers or the training recipe. YOLOv12n [19] is not included in this controlled study because its A2C2f attention design would require a separate adaptation strategy. Models without a stable implementation in the same Ultralytics workflow, including YOLOv14-style variants, are omitted for reproducibility.

A. Reference YOLOv11 Architecture

Figure 1 gives the reference layout used in the controlled comparison. For clarity, the diagram is read from left to right: the early blocks encode orchard-image patterns, the middle route exchanges information between resolutions, and the final prediction layers convert the fused maps into boxes and class confidence values.

In this baseline, Conv and C3k2 units form the main visual encoder. SPPF then broadens the local context, while C2PSA adjusts the channel responses before the features enter the neck. The neck passes information in both directions across the pyramid, allowing fine spatial detail and coarse semantic cues to be combined before detection.

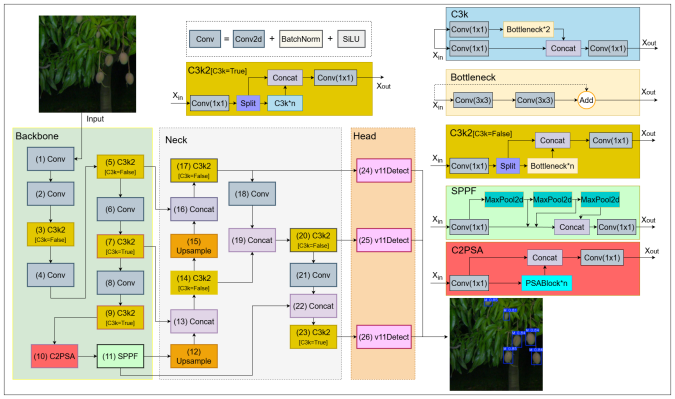


Fig. 1: Reference YOLOv11 diagram used in the experiments, including the module list attached to the layout.

B. Ghost-Based YOLOv11 Variant

Ghost-YOLOv11n preserves the YOLOv11 [9] forward route and changes only selected feature-producing layers. The first Conv layer is left unchanged, subsequent Conv layers in the backbone and head use GhostConv, and each C3k2 unit is replaced with C3k2Ghost. SPPF, C2PSA, upsampling, concatenation, and Detect operations are retained. Figure 2 illustrates the resulting detector, which keeps the topology fixed while changing the internal feature-generation mechanism.

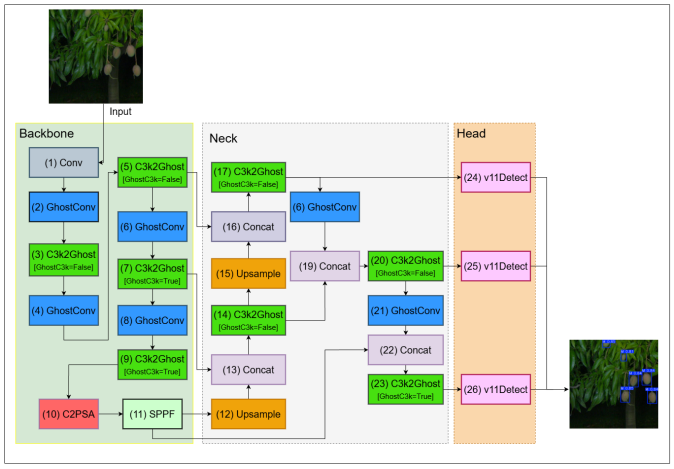


Fig. 2: Ghost-YOLOv11n layout after replacing selected YOLOv11n feature blocks with GhostConv and C3k2Ghost modules.

C. Ghost Module Design

The proposed detector relies on three Ghost-related components. Their roles are described below.

1) The GhostConv Module: Let a denote the number of output spatial locations, s_k the kernel area, and d_{in} and d_{out} the input and output channel widths. The leading multiply-add term of a dense convolution can be written as:

$$\Omega_{\text{dense}} = a\, s_k\, d_{in}\, d_{out}. \tag{1}$$

When neighboring mango regions produce similar activations, part of this cost may be spent on maps that carry overlapping information. GhostConv reduces this expense by generating only a subset of maps with a regular convolution and deriving the rest with cheaper depth-wise operations.

For a target width d_{out}, the dense branch first creates $d_g = d_{out}/2$ intrinsic maps with the kernel and stride of the replaced layer:

$$U = G_{\text{main}}(X), \qquad U \in \mathbb{R}^{d_g \times h \times w}. \tag{2}$$

The cheap branch then transforms these intrinsic maps with a depth-wise 5×5 operator:

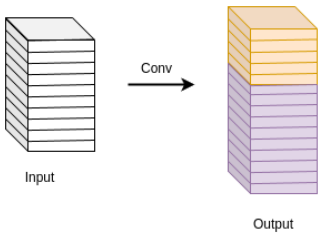
$$V = G_{\text{cheap}}(U). \tag{3}$$

The module output is

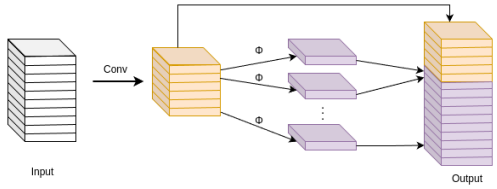
$$Y = \text{cat}_{ch}(U, V), \tag{4}$$

so the block returns d_{out} channels while only half of them are produced by the dense branch. In the Ultralytics implementation, both branches use the Conv wrapper, including convolution, BatchNorm, and SiLU unless activation is disabled by the surrounding block.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 19:25:00 UTC from IEEE Xplore. Restrictions apply.



(a) The standard convolutional layer.



(b) The GhostConv module.

Fig. 3: Conventional convolution and GhostConv. GhostConv produces intrinsic maps first and then expands the channel set through a depth-wise branch.

2) The GhostBottleneck Module: The GhostBottleneck block [14], shown in Figure 4, places two GhostConv layers around a shortcut. The first layer increases channel width and the second restores the requested output width. For downsampling, an additional depth-wise convolution is placed on the main path; without downsampling, the shortcut is an identity connection.

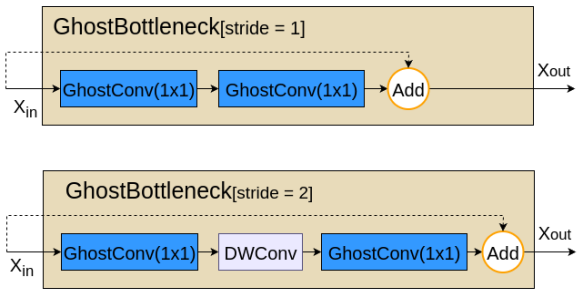


Fig. 4: GhostBottleneck structures used with and without spatial downsampling.

3) The C3k2Ghost Module: C3k2Ghost is the Ghost-based counterpart of the C3k2 aggregation unit. As illustrated in Figure 5, a 1×1 GhostConv first expands the input and divides it into two hidden routes. The transformed route uses GhostBottleneck or GhostC3k components according to the original C3k2 setting, and a final 1×1 GhostConv fuses the hidden features. The split-merge pattern is therefore preserved, but the internal transformations use cheaper Ghost operations.

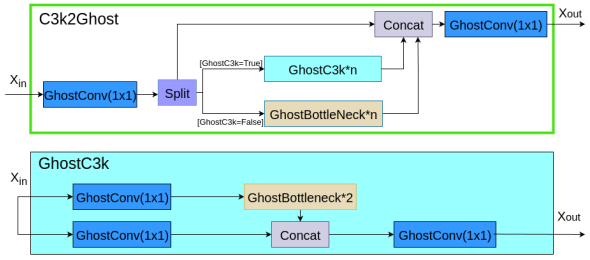


Fig. 5: C3k2Ghost data flow, showing the GhostC3k branch and the retained split-merge aggregation route.

IV. Experiments

A. Dataset

MangoYOLO [10] contains 1,730 annotated mango images. Before augmentation, the collection was locked into three disjoint groups: 1,212 for model fitting, 346 for validation, and 172 reserved for final testing. Augmentation touched only the training images. Roboflow was used to create three offline variants of each training sample through flipping, right-angle rotation, cropping, and brightness or saturation perturbation. The validation and test pools stayed in their original form. After this step, the experiment used 4,154 images overall, distributed as 3,636/346/172 for train/validation/test. All images were resized to 512×512 before they were passed to the detector.

B. Experimental Setup

All compared detectors were evaluated in the same PyTorch and Ultralytics environment on one NVIDIA RTX5090 GPU. YOLOv8n, YOLOv10n, YOLOv11n, YOLOv11m, and Ghost-YOLOv11n were trained from random initialization for 150 epochs. The training setting used 512×512 inputs, batch size 16, Adam optimization, learning rate 10^{-3}, and early-stopping patience of 30 epochs. Inference time was measured on the 172 test images with batch size 1. A dummy pass was executed before timing, CUDA synchronization was applied during measurement, and the average forward latency was converted to FPS by $1000/\text{latency}$. The timing excludes image preprocessing and NMS post-processing.

C. Evaluation Metrics

The evaluation reports detector quality, model cost, and image-level counting error.

1) Detection Performance Metrics: After IoU matching, denote the matched detections, unmatched predicted boxes, and missed annotations by n_{tp}, n_{fp}, and n_{fn}, respectively. Box-level precision and recall are expressed as:

$$\text{Prec}_{box} = \left(1 + \frac{n_{fp}}{n_{tp}}\right)^{-1} \tag{5}$$

$$\text{Rec}_{box} = \left(1 + \frac{n_{fn}}{n_{tp}}\right)^{-1} \tag{6}$$

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 19:25:00 UTC from IEEE Xplore. Restrictions apply.

TABLE I: MangoYOLO test results under the shared counting threshold. Bold numbers mark the best result within each metric group.

Model	Params (M) ↓	GFLOPs ↓	Latency (ms) ↓	FPS ↑	AP50 (%) ↑	AP50:95 (%) ↑	Count MAE ↓	Count RMSE ↓
YOLOv8n	3.01	4.10	**1.82**	**548.9**	98.23	70.70	0.919	1.402
YOLOv10n	2.71	4.20	2.48	402.8	98.24	71.45	0.936	1.416
YOLOv11n (Base)	2.59	3.22	2.44	410.3	98.15	71.31	0.884	1.351
YOLOv11m	20.05	34.09	3.41	293.1	98.12	**71.63**	0.814	**1.276**
Ghost-YOLOv11n	**1.85**	**2.70**	2.36	422.9	**98.38**	70.34	**0.773**	1.287

The detection table also reports AP50. The stricter AP50:95 score is computed by taking AP at ten IoU cutoffs, from 0.50 to 0.95 with a 0.05 step, and then averaging those values.

2) Efficiency and Counting Metrics: Efficiency is reported through trainable parameters, GFLOPs, latency, and FPS. For counting, the ground-truth and predicted mango counts of test image i are denoted by y_i and $\hat{y}_i$. We define $r_i = \hat{y}_i - y_i$ and use an overbar to indicate averaging across the N test images:

$$\mathrm{MAE}_{cnt} = \overline{|r_i|} \tag{7}$$

$$\mathrm{RMSE}_{cnt} = \sqrt{\overline{r_i^2}} \tag{8}$$

RMSE assigns a larger penalty to images with large count errors.

D. Threshold Calibration for Counting Task

The final count is affected by both the confidence cutoff and the overlap threshold used in NMS. These two values were calibrated before test evaluation rather than chosen from test-set behavior. We performed a grid search on the validation split with YOLOv11n and selected the pair that minimized validation MAE (Figure 6). The chosen operating point was confidence 0.55 with NMS IoU 0.25. This same pair was then applied to all test-set models, so the reported MAE and RMSE correspond to one shared deployment setting.



Fig. 6: Validation grid used to select the shared confidence and NMS IoU setting for count evaluation.

E. Results

Figure 7 reports the Ghost-YOLOv11n learning curves. The loss terms fall sharply in the early epochs and then flatten, while validation mAP@50–95 increases and later stabilizes. This behavior indicates that the training run converged on the MangoYOLO split.

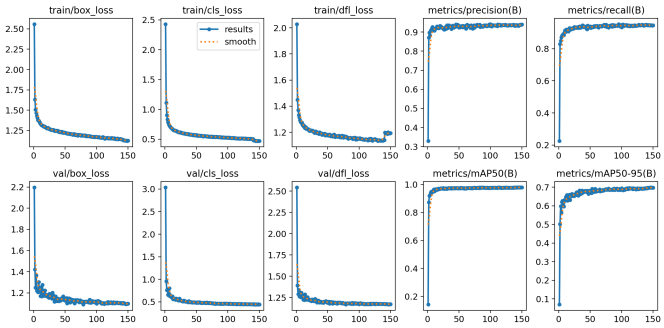


Fig. 7: Ghost-YOLOv11n loss and validation-metric curves during the 150-epoch training run.

1) Comparison with YOLO Baselines: Table I compares the YOLO variants on the test split under the training and threshold protocol described above.

Ghost-YOLOv11n is smaller than YOLOv11n, reducing the parameter count from 2.59M to 1.85M and computation from 3.22 to 2.70 GFLOPs. The measured forward latency also decreases from 2.44 ms to 2.36 ms. Although AP50:95 is lower, AP50 rises to 98.38%, and the count error is the best among the tested detectors. With the fixed threshold pair, Ghost-YOLOv11n obtains 0.773 MAE; YOLOv11n and YOLOv11m obtain 0.884 and 0.814, respectively.

2) Ablation Study: Table II separates the effect of replacement location from the effect of module family. In the table, B denotes backbone, N denotes neck, GC denotes GhostConv, C3 denotes C3k2Ghost, M denotes parameter count in millions, and G denotes GFLOPs.

TABLE II: Ablation results for replacement location and Ghost module family.

Variant	B	N	GC	C3	M ↓	G ↓	FPS ↑	AP50:95 ↑	MAE ↓
YOLOv11n	-	-	-	-	2.59	3.22	410.3	**71.31%**	0.884
Backbone only	✓	-	✓	✓	2.04	**2.58**	415.8	70.74%	0.936
Neck only	-	✓	✓	✓	2.08	2.71	404.0	70.50%	0.837
GhostConv only	✓	✓	✓	-	2.26	2.83	387.9	71.06%	0.843
C3k2Ghost only	✓	✓	-	✓	2.17	3.09	**462.5**	70.07%	0.779
Ghost-YOLOv11n	✓	✓	✓	✓	**1.85**	2.70	422.9	70.34%	**0.773**

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 19:25:00 UTC from IEEE Xplore. Restrictions apply.

The ablation results indicate that the two Ghost replacements do not behave identically. Using Ghost modules only in the backbone increases MAE to 0.936, suggesting that early features should not be compressed too aggressively. Replacing only standard convolutions reduces nominal computation, but it does not improve measured GPU speed, which can happen when depth-wise operators become memory-bound. The C3k2Ghost-only variant reaches the highest FPS, 462.5, and remains close to the best count error with 0.779 MAE. Combining GhostConv and C3k2Ghost gives the lowest MAE, 0.773, while maintaining real-time speed.

3) Qualitative Analysis: Figure 8 is included to examine two counting failures that are easy to miss when only scalar scores are reported. In the first row, Ghost-YOLOv11n assigns a box to a leaf-covered mango that YOLOv11m does not mark. In the second row, the proposed detector avoids treating a foliage patch as fruit, which is consistent with its smaller count error in Table I.

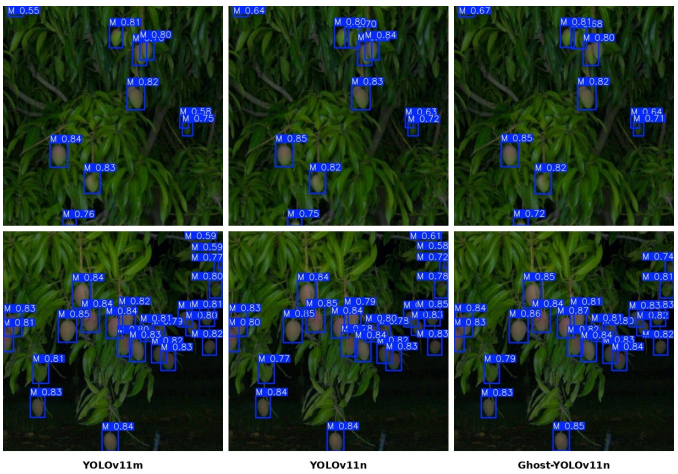


Fig. 8: Two visual test cases illustrating occluded-fruit recovery and suppression of foliage-induced false boxes.

V. Conclusion

Ghost-YOLOv11n offers a compact YOLOv11n-based detector for mango counting in crowded orchard images. The redesign changes selected feature-generation blocks to GhostConv and C3k2Ghost while keeping the multi-scale detection path unchanged.

The experimental results show that this targeted substitution reduces both storage and computation. Relative to YOLOv11n, the proposed model lowers parameters by 28.6% and GFLOPs by 16.1%, while the measured forward latency is 2.36 ms per image. With the validation-selected threshold pair applied to every model, Ghost-YOLOv11n achieves the best counting score in the comparison, with 0.773 MAE.

The ablation study further suggests that improving the count is not identical to maximizing strict localization metrics such as mAP@50–95. In dense mango scenes, the most useful compression occurs inside CSP-style aggregation blocks rather than through uniform compression of all stages. Future experiments should examine per-model threshold calibration, Ghost adaptations for YOLOv12-style architectures, INT8 deployment, and transfer to other dense fruit-counting datasets.

References

[1] A. Kamilaris and F. X. Prenafeta-Boldú, "Deep learning in agriculture: A survey," *Computers and Electronics in Agriculture*, vol. 147, pp. 70–90, 2018. [Online]. Available: https://doi.org/10.1016/j.compag.2018.02.016

[2] L.-D. Quach, K. N. Quoc, A. N. Quynh, and H. T. Ngoc, "Evaluation of the efficiency of the optimization algorithms for transfer learning on the rice leaf disease dataset," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 13, no. 10, 2022.

[3] K. H. Nguyen, H. V. N. Nguyen, H. N. Tran, and L.-D. Quach, "Combining autoencoder and yolov6 model for classification and disease detection in chickens," in *Proceedings of the 2023 8th International Conference on Intelligent Information Technology*, 2023, pp. 132–138.

[4] Y. Tian, G. Yang, Z. Wang, H. Wang, E. Li, and Z. Liang, "Apple detection during different growth stages in orchards using the improved YOLO-V3 model," *Computers and Electronics in Agriculture*, vol. 157, pp. 417–426, 2019. [Online]. Available: https://doi.org/10.1016/j.compag.2019.01.012

[5] H. T. Ngoc, N. N. Vinh, N. T. Nguyen, and L.-D. Quach, "Efficient evaluation of slam methods and integration of human detection with yolo based on multiple optimization in ros2," *International Journal of Advanced Computer Science and Applications (IJACSA)*, vol. 14, no. 11, 2023.

[6] H. K. Hua, K. H. Nguyen, L.-D. Quach, and H. N. Tran, "Traffic lights detection and recognition method using deep learning with improved yolov5 for autonomous vehicle in ros2," in *Proceedings of the 2023 8th International Conference on Intelligent Information Technology*. ACM, 2023, pp. 117–122.

[7] X. Lin, D. Liao, Z. Du, B. Wen, Z. Wu, and X. Tu, "SDA-YOLO: An object detection method for peach fruits in complex orchard environments," *Sensors*, vol. 25, no. 14, p. 4457, 2025. [Online]. Available: https://doi.org/10.3390/s25144457

[8] K. Han, Y. Wang, Q. Tian, J. Guo, C. Xu, and C. Xu, "Ghostnet: More features from cheap operations," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2020, pp. 1580–1589. [Online]. Available: https://doi.org/10.48550/arXiv.1911.11907

[9] R. Khanam and M. Hussain, "Yolov11: An overview of the key architectural enhancements," *arXiv preprint arXiv:2410.17725*, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2410.17725

[10] A. Koirala, K. Walsh, Z. Wang, and C. McCarthy, "Mangoyolo dataset," 2021, last accessed: 2025/07/01. [Online]. Available: https://acquire.cqu.edu.au/articles/dataset/MangoYOLO_data_set/13450661

[11] M. Stein, S. Bargoti, and J. Underwood, "Image based mango fruit detection, localisation and yield estimation using multiple view geometry," *Sensors*, vol. 16, no. 11, p. 1915, 2016. [Online]. Available: https://doi.org/10.3390/s16111915

[12] N. T. Anderson, K. B. Walsh, A. Koirala, Z. Wang, M. H. Amaral, G. R. Dickinson, P. Sinha, and A. J. Robson, "Estimation of fruit load in australian mango orchards using machine vision," *Agronomy*, vol. 11, no. 9, p. 1711, 2021. [Online]. Available: https://doi.org/10.3390/agronomy11091711

[13] N. Tamrakar, S. Karki, M. Y. Kang, N. C. Deb, E. Arulmozhi, D. Y. Kang, J. Kook, and H. T. Kim, "Lightweight improved YOLOv5s-CGhostnet for detection of strawberry maturity levels and counting," *AgriEngineering*, vol. 6, no. 2, pp. 962–978, 2024. [Online]. Available: https://doi.org/10.3390/agriengineering6020055

[14] K. Han, Y. Wang, Q. Tian, J. Guo, C. Xu, and C. Xu, "Ghostnet: More features from cheap operations," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2020, pp. 1580–1589. [Online]. Available: https://doi.org/10.1109/CVPR42600.2020.00165

[15] K. Han, Y. Wang, C. Xu, J. Guo, C. Xu, E. Wu, and Q. Tian, "Ghostnets on heterogeneous devices via cheap operations," *International Journal of Computer Vision*, vol. 130, pp. 1050–1069, 2022. [Online]. Available: https://doi.org/10.1007/s11263-022-01575-y

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 19:25:00 UTC from IEEE Xplore. Restrictions apply.

[16] J. Sun, Y. Peng, C. Chen, B. Zhang, Z. Wu, Y. Jia, and L. Shi, "ESC-YOLO: optimizing apple fruit recognition with efficient spatial and channel features in YOLOX," *Journal of Real-Time Image Processing*, vol. 21, p. 162, 2024. [Online]. Available: https://doi.org/10.1007/s11554-024-01540-7

[17] L. He, F. Wu, X. Du, and G. Zhang, "Cascade-SORT: A robust fruit counting approach using multiple features cascade matching," *Computers and Electronics in Agriculture*, vol. 200, p. 107223, 2022. [Online]. Available: https://doi.org/10.1016/j.compag.2022.107223

[18] Y. Tang, J. Qiu, Y. Zhang, D. Wu, Y. Cao, K. Zhao, and L. Zhu, "Optimization strategies of fruit detection to overcome the challenge of unstructured background in field orchard environment: a review," *Precision Agriculture*, vol. 24, pp. 1183–1219, 2023. [Online]. Available: https://doi.org/10.1007/s11119-023-10009-9

[19] Y. Tian, Q. Ye, and D. Doermann, "YOLOv12: Attention-centric real-time object detectors," *arXiv preprint arXiv:2502.12524*, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2502.12524

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 25,2026 at 19:25:00 UTC from IEEE Xplore. Restrictions apply.